

The Bottleneck Moves Twice: Three Regimes of On-Device LLM Latency

Bapista Khan
Collab-Foundry
bapistakhan@gmail.com

ABSTRACT

On-device language models are deployed for privacy, offline operation and cost, yet the latency intuition the community carries is inherited from datacentre GPU serving, where autoregressive *decode* dominates. Prior work has characterised inference on datacentre CPUs with matrix accelerators and HBM, and quantisation effects on high-end Apple Silicon. We ask a question neither answers: **across the commodity hardware an organisation actually owns, how much of latency behaviour is a property of the machine, and how much is a property of the workload?** We instrument end-to-end inference across four on-device platforms—two x86 CPU vendors, an ARM single-board computer, and 8 GB Apple Silicon—and find that **beyond ~1,400 prompt tokens the prefill share of end-to-end latency converges to within two percentage points across all non-accelerated devices**, despite absolute latency differing by up to 14× and model size by 4.7×. At ~1,400 prompt tokens every such device sits at 91–93% prefill; by ~2,440 tokens the three lie within 1.3 points of one another. A same-model control isolates the effect: identical weights, 14.4× apart in prefill time, two points apart in share. The bottleneck then moves a second time. On 8 GB Apple Silicon a 3B model **re-inverts to decode-bound** beyond ~2,029 tokens as decode throughput collapses from 49 to 17 tok/s, while smaller models on the same device never do. The collapse is measured; its cause is not yet isolated, and we state what our data rules out. The share is also **predictable**: one calibration of the prefill-to-decode rate ratio predicts it to within 2.6 points from ~330 prompt tokens upward, until decode collapses—and then the prediction fails by 17 points, so a runtime can **detect the collapse from its own timings**. We argue this is the missing precondition for regime-aware scheduling across heterogeneous owned devices, and that single-number throughput reporting conceals it. Harnesses and raw data are released.

KEYWORDS

on-device inference, mobile systems, LLM, latency, prefill, decode, measurement

1 INTRODUCTION

Running language models on the device rather than in the datacentre is no longer exotic. Privacy requirements, offline operation, and the cost of inference at scale have pushed capable models onto phones, laptops and edge hardware [10, 16, 17]. The engineering intuition that travelled with them did not come from those devices. It came from GPU serving, where the dominant cost is autoregressive *decode* and where optimisation effort—batching, paged attention, speculative decoding—has been directed accordingly [7, 8].

We took that intuition to four on-device platforms and measured it. It does not merely fail. It fails *differently at different context lengths on the same machine*.

On an 8 GB Apple Silicon device, a 3B model is **decode-bound** at short context—prefill accounts for only 20% of end-to-end latency. As context grows it becomes **prefill-bound**, peaking at 80% near 1,500 tokens. Then, past roughly 2,029 tokens, it **re-inverts**: prefill’s share falls back to ~69% while decode throughput collapses from 49 to 17 tok/s. Nothing changed but the length of the prompt.

Smaller models on the same device never reach that third regime—0.5B and 1.5B climb monotonically and their decode throughput declines only gently. A CPU-only machine sits at the other extreme: it crosses into the prefill-bound regime within ~200 tokens and never leaves it, climbing to 95%.

So the number of regimes a deployment traverses is not a property of the platform alone: on one device it depends on which model the developer chose. Our hypothesis is that it is set by **memory headroom**, which would make it a design-time choice; Section 3 states what our data does and does not support.

What has already been measured. Recent studies bracket this space; none answers the question above. Na et al. [11] characterise LLM inference on *datacentre* CPUs with matrix extensions and high-bandwidth memory, asking whether such machines can stand in for GPUs. Benazir and Lin [2] profile *high-end* Apple Silicon from a quantization perspective. Zhang and Huang [18] report that a mobile CPU can outperform the same device’s GPU, but report throughput alone—without separating prefill from decode, or varying context. Zhao et al. [19] *optimise* CPU serving through

configuration search and hot-switching. Hydra [15] separates the phases on GPU-accelerated Jetson SoCs, where a 500-token output budget makes decode dominate, and names longer contexts and memory-capacity pressure as future work. Our question is orthogonal to all five: on the commodity, non-accelerated hardware an organisation already owns, how much of latency behaviour belongs to the machine and how much to the workload—and we characterise rather than optimise.

Contributions.

- We identify **three distinct latency regimes** for on-device inference and show a single device traversing all three as context grows (§2).
- We show which regimes a configuration reaches is not fixed by platform class (§2), derive the cross-machine invariance and its limits, and bound the cause of the collapse (§3).
- We show that one calibration measurement predicts the prefill share within 2.6 points, and that the prediction’s failure detects the collapse (§3.2).
- We draw out what follows for benchmarking, scheduling and context budgeting, and argue that single-number throughput reporting conceals every transition (§4).
- We release both harnesses and all raw data (§6).

2 THREE REGIMES

Setup. The Platform A and B studies sweep prompt length from 55 to 2,336 tokens with five repetitions per point, discard warm-up runs, and time prefill and decode separately. Output is capped at 64 tokens: MLX generates all 64, while the Ollama runs stop at end-of-sequence, after 10–37 tokens on Platform A. Platform A is a CPU-only machine (AMD Ryzen 7 8845HS, 8 cores / 16 threads, 14 GiB) running quantized Qwen2.5-7B, Llama-3.1-8B and Gemma2-9B through Ollama’s llama.cpp backend [6]. Platform B is an 8 GB MacBook Air running, on its GPU, 4-bit MLX builds of Qwen2.5-Instruct at 0.5B, 1.5B and 3B—a deliberate size sweep on a device where larger families do not fit. To test whether the CPU-side behaviour is a property of one machine, we add two further CPU-only devices, swept at four prompt lengths from 121 to 2,439 tokens: Platform C, an Intel Core i7-10700T (Comet Lake, 16 threads, 62.6 GiB), and Platform D, a Raspberry Pi 5 (quad-core Arm Cortex-A76, 15.6 GiB). Platforms A and C run Qwen2.5-7B; Platform D runs Qwen2.5-1.5B, with a same-model 7B control reported below.

Table 2 is the paper. A developer who profiles this configuration at 600 tokens and one who profiles it at 2,300 tokens will reach opposite conclusions about what to optimise, on identical hardware running an identical model.

Table 1: The three regimes. The binding constraint changes, and with it the only lever that helps.

Regime	Signature	Bound by	Lever
1. Decode-bound	prefill <50%	sequential cost	output length
2. Prefill-bound	prefill rising, >50%	compute	context length
3. Collapse	prefill <i>falling</i> , decode collapsing	unresolved	model size

Table 2: One device, three regimes. Qwen2.5-3B (4-bit) on an 8 GB Apple Silicon machine: prefill share rises, peaks, then *reverses* as decode collapses. Means over five repetitions.

Context (tokens)	Prefill (% of latency)	Decode (tok/s)	Regime
55	19.9	49.5	1 · decode-bound
140	35.4	49.2	1
300	50.5	49.2	→ 2 · crossover
600	64.6	49.2	2 · prefill-bound
1000	74.5	48.4	2
1500	79.9	43.5	2 · peak
2000	68.8	23.5	→ 3 · collapse
2336	69.4	17.3	3 · collapse

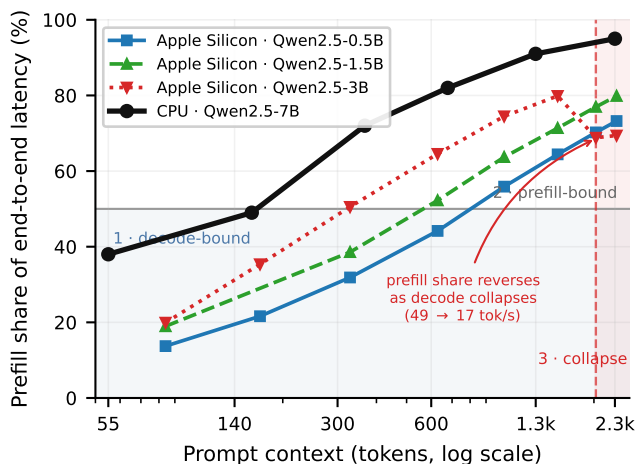


Figure 1: Prefill’s share of end-to-end latency against context length. The CPU (black) crosses into regime 2 early and never leaves it, reaching 95%. On Apple Silicon the smaller models climb and level off, while Qwen2.5-3B peaks near 1,500 tokens and then *reverses*—not because prefill became cheap, but because decode collapsed. Means over five repetitions; one flagged 1.5B outlier excluded.

Model size decides how far you travel. On the same device, Qwen2.5-0.5B climbs monotonically from 13.7% to 73.2%

prefill with decode declining only gently (250 to 208 tok/s), and 1.5B behaves the same way (19.0% to 79.9%; 96 to 87 tok/s).¹ Neither reaches regime 3. Only the 3B model, the largest the device runs, re-inverts.

Some platforms never leave regime 2. Platform A crosses into regime 2 just past 160 tokens and shows no reversal anywhere in the sweep. Prefill climbs monotonically—38% at 55 tokens (an 11-token answer), 72% at 367, 91% at 1,299, **95%** at 2,336—while decode sits flat at 11–12 tok/s. The machine simply lacks the parallelism to make prefill cheap. Llama-3.1-8B and Gemma2-9B reproduce this at 95–96% prefill.

The crossover between regimes 1 and 2 also moves with model size, and in the direction that surprises people: *earlier* for larger models—~830 tokens at 0.5B (interpolated) against ~330 at 3B. Bigger models reach the prefill-bound regime sooner, and only the biggest reaches regime 3.

Quantization does not behave as the folklore says—as others have also found. Benazir and Lin [2] report that lower bit-width does not reliably mean lower latency on Apple Silicon, and give the mechanism. We observe the same ordering on commodity CPU-only hardware: on Platforms A and C prefill *speed* is non-monotonic in footprint, with the 8-bit build prefilling fastest and an intermediate K-quant slowest, inverting the “quantize down for speed” heuristic [4, 5, 9]. **We report this as corroboration on a different hardware class, not as a new result.** What smaller reliably buys is *headroom*, our leading candidate for what keeps a deployment out of regime 3.

The share is the same across machines—once the context is long enough. Table 3 sweeps Platforms A, C and D together. Two instruction-set architectures and a 4.7× model-size difference separate them, and absolute cost differs by more than an order of magnitude, yet at long context the prefill *share* converges: 2.0 points apart at ~1,400 tokens and 1.3 points at ~2,400. **The convergence is a long-context property and we claim it only there.** At the shortest point the three machines are 7.3 points apart, and Platform D’s short-context variance is itself large ($\sigma = 16.0$); at 121 prompt tokens the fixed costs of a request are a meaningful share of it.

A same-model control isolates the effect. Model size is not held constant in Table 3, so we ran identical weights—Qwen2.5-7B, Q4_K_M, 1,402 prompt tokens—on all three. Wall-clock prefill was 30.08 s on Platform A, 83.54 s on C and 431.72 s on D: a **14.4× spread**. The prefill share was 92%, 93% and 94%. **Fourteen times the latency, two points of share.**

¹A 1.5B point near 140 tokens is excluded: it stopped after 13 of 64 tokens.

Table 3: Prefill share (%) at matched prompt lengths on three non-accelerated devices. Convergence is a long-context property: 7.3 points apart at 121 tokens, 1.3 points apart at 2,439. Five repetitions per point.

Prompt (tok)	A	C	D	Spread
	Zen4, 7B	C.Lake, 7B	A76, 1.5B	
121	43.9	51.2	51.0	7.3
453	83.3	85.4	79.1	6.3
1,403	91.4	93.2	91.2	2.0
2,439	95.6	96.4	96.9	1.3

Threats to validity. Platform C was measured on 15 July 2026 and Platforms A and D on 13 August, and the runtime version differs across the three (Ollama 0.19.0, 0.17.0 and 0.17.7 respectively). We report this because the claim is a cross-machine one. Neither confound plausibly explains convergence *towards* a common value as context grows—a version artefact would bias a machine roughly uniformly rather than vanish with prompt length—but a controlled re-run on a single runtime version would close the question, and we have not done it. Platforms A, C and D are CPU-only; Platform B uses the Apple GPU and is excluded from the invariance claim. The cross-machine sweep logged phase durations but not output token counts.

3 WHY THE BOTTLENECK MOVES

3.1 Why the share is invariant

The invariance of Section 2 is not a coincidence; it follows from the definition of the quantity. Write N_p for prompt tokens, N_o for generated tokens, and R_p, R_d for the platform’s prefill and decode rates in tokens per second. The prefill share of end-to-end latency is

$$\phi = \frac{N_p/R_p}{N_p/R_p + N_o/R_d} = \frac{1}{1 + \rho \frac{N_o}{N_p}}, \quad \rho \equiv \frac{R_p}{R_d}. \quad (1)$$

Only two quantities appear: the workload shape N_o/N_p , and ρ , the *ratio* of the two rates. Absolute speed does not. If one platform is k times faster than another at everything— $R_p \rightarrow kR_p$ and $R_d \rightarrow kR_d$ —then ρ is unchanged and so is ϕ . Two machines whose wall-clock differs by an order of magnitude therefore report the same share, which is precisely what Section 2 measures: a 14× spread in latency and a two-point spread in ϕ .

The same equation has a less flattering consequence: as N_p/N_o grows, $\phi \rightarrow 1$ for *every* ρ , so some convergence at long context is guaranteed by arithmetic alone. The informative quantity is therefore ρ itself. For the 7B model at 1,403 tokens the prefill-to-decode time ratio is 10.6 on Platform A

and 13.7 on C: within 30%, while prefill time differs 2.8 $\times$. This assumes equal output length, which identical weights at temperature zero make likely but our logs do not record.

Equation 1 also says when the invariance *must* fail: whenever ρ is not constant across platforms. That gives two predictions, and the rest of this section is their two instances.

Prediction 1: accelerators that parallelise prefill. A GPU raises R_p far more than R_d , since prefill is parallel across tokens while decode is sequential and bandwidth-bound. Larger ρ means smaller ϕ : the platform looks decode-bound. *This is the datacentre intuition, and Equation 1 reproduces it as a special case rather than a contradiction.* Platform B’s integrated GPU does not show it strongly at batch size one: its ρ of 5–6 is close to Platform A’s 4.0–4.4. Because its runs also generate a full 64 tokens, we claim invariance only across the CPU-only platforms.

Prediction 2: collapse of R_d . If decode throughput falls sharply while prefill is unaffected, ρ rises and ϕ turns back down—not because prefill got cheaper, but because decode got worse. The regime 2 $\rightarrow$ 3 transition below is exactly this.

Two transitions follow; we can explain the first.

Regime 1 $\rightarrow$ 2. Prefill is a single pass over the context and is parallel across tokens; it is compute-bound. Decode is sequential—each token depends on the last—and is bound by memory bandwidth, streaming weights and a growing KV cache at every step [1, 7]. Prefill cost therefore scales with context while decode cost per token does not, so prefill eventually overtakes. On hardware that parallelises prefill well this takes a long prompt; on a CPU it happens almost immediately, which is why Platform A crosses within $\sim$ 160 tokens. The datacentre disaggregation literature separates the two phases for exactly this reason [13, 21]; on-device there is nowhere to disaggregate to.

Regime 2 $\rightarrow$ 3. Past $\sim$ 1,500 tokens the 3B model’s decode throughput falls from 43 to 17 tok/s while its prefill throughput falls only 16%. Because prefill’s share is a *ratio*, decode getting slower makes prefill look smaller and the curve turns back down: Prediction 2. The reversal is not prefill becoming cheap. It is decode collapsing. **What collapses it we have not isolated**, and our data rules out the obvious answer: MLX’s own peak allocation is 2.44 GB of the 8 GB and stays flat across the collapse, and the 3B model’s KV cache at 2,336 tokens is under 100 MB, so the model’s footprint alone does not exhaust memory. Two candidates remain. One is system-wide memory pressure on an 8 GB machine that also runs an operating system. The other is thermal limits on a fanless chassis: our runs went in ascending prompt length, so the longest prompts were also the latest. Only the largest model collapses, which

Table 4: Equation 1 as a predictor. ρ calibrated once per configuration; error in points of prefill share at the other measured lengths.

Configuration	ρ	max err , 330–1,530	err $\geq$ 2,000 tok
A $\cdot$ CPU $\cdot$ 7B	4.14	1.1	−0.1
B $\cdot$ GPU $\cdot$ 0.5B	12.42	2.6	+1.6, +1.6
B $\cdot$ GPU $\cdot$ 1.5B	8.99	2.2	+1.0, +0.6
B $\cdot$ GPU $\cdot$ 3B	5.40	1.8	+16.6, +17.9

fits both. Separating them needs a randomised-order rerun with swap and thermal telemetry.

This is why the three are regimes rather than points on a curve. In regime 1 the lever is output length; in regime 2 it is context length; in regime 3 neither helps, and on our data only a smaller model avoids it. An optimisation that works in one is neutral or harmful in another.

3.2 One measurement predicts the share; its failure flags the collapse

Equation 1 is a definition, but it becomes a prediction if ρ is stable. We calibrate ρ once per configuration, at $\sim$ 650 tokens, and predict ϕ at every other measured length from that ρ and the point’s own token counts (Table 4). From $\sim$ 330 tokens until decode collapses, the error never exceeds 2.6 points across 16 predictions on two platforms (mean 1.0). Below $\sim$ 300 tokens it degrades—up to 15 points on Platform A at 55 tokens—where fixed per-request costs distort both rates.

On the 3B model the prediction fails by **17 points** at 2,029 and 2,365 tokens, six times its worst error elsewhere above 300 tokens. That failure is useful: a runtime that calibrates ρ once and watches the residual between predicted and observed share *detects the collapse from its own timings*, without a model of its cause. Predictions use each point’s realised output length. Platforms C and D did not log it and are omitted.

4 WHAT THIS MEANS FOR MOBILE SYSTEMS

1. *Single-number benchmarks conceal every transition.* Reporting “tokens per second” collapses two phases with opposite scaling into one figure, and says nothing about which regime produced it. Two devices reporting the same figure can behave completely differently as a user’s context grows. On-device benchmarks should report prefill and decode separately at minimum, and prefill *fraction against context length* by preference—a curve, not a number. The shape of that curve is the diagnostic: monotonic means two regimes, rise-then-fall means three.

2. *Profiling at one context length is profiling one regime.* Table 2 makes this concrete: the same configuration profiled at 600 and at 2,300 tokens yields opposite optimisation advice. Any measurement reported without its context length is close to meaningless.

3. *Placement and offload decisions inherit the wrong prior.* Routing and cascade systems typically cost a request by expected output length [3, 12, 14]. That is right in regime 1, wrong in regime 2, and dangerously wrong in regime 3, where the true cost is that the request has pushed the device past a cliff it will not recover from until the context shrinks.

4. *The collapse is a design-time constraint.* Because it is a discontinuity rather than a slope, on-device capability is not gracefully degradable. An assistant that behaves well carrying 1,800 tokens of history may be unusable at 2,200 on the same phone. That determines how much conversation history a product can keep—an architecture decision, not a tuning flag. And since only the largest model collapsed, choosing a smaller model may be better understood as buying regime avoidance than as buying speed.

5. *Context budgeting is the largest available lever in regime 2.* If latency is governed by what the model reads, then what it reads is the thing to control. We demonstrated this on Platform A: gating the costliest optional stage of an agent pipeline—a second “review/verify” model pass—cut end-to-end latency 2.4× with no measurable quality loss under an order-swapped LLM judge [20] ($n = 8$: seven ties, one win for the second pass). It is the largest single lever on that platform, and it is invisible to anyone optimising decode.

5 OPEN QUESTIONS

Can a runtime detect which regime it is in, and adapt? The three-regime picture makes this tractable. The signals are cheap and already available at inference time—prefill fraction, decode throughput, memory headroom—and each regime has a known corresponding action: shorten output in regime 1, budget context in regime 2, move to a smaller model in regime 3. This is not phase hot-switching [19]: the phase is known when a request arrives, whereas the regime depends on context length and headroom and can change mid-session. A runtime that classified its own regime and switched policy would need no platform-specific tuning from the developer. Section 3.2 supplies the detector: the residual of Equation 1. Whether such a controller is stable near the boundaries, and whether the regime-3 transition can be predicted rather than merely detected after the collapse, we do not know.

Is there a fourth regime? None of our platforms uses a dedicated neural accelerator. An NPU may add one, or may sit

inside an existing regime. If regimes keep accumulating, the argument for measuring rather than assuming strengthens.

Do architectural changes remove a regime or add one? Mixture-of-experts alters what must be resident during decode; speculative decoding attacks the sequential dependency that makes decode slow [8]. Both act directly on the mechanisms above. Whether they collapse regimes together or separate them further is empirical, and we have not answered it.

Limitations. Each platform is a single machine; three CPUs across two instruction sets are the cross-device check. We measure three model sizes on Platform B and three families on Platform A, at one primary quantization level each. Absolute figures are characteristic of these devices. The claim we defend is the *structure*: three regimes and two transitions, measured. That memory headroom governs the second is a hypothesis we have not yet isolated.

6 CONCLUSION

On-device LLM latency has no single characterisation, and not merely because devices differ. The bottleneck moves *within* a device: decode-bound at short context, prefill-bound through the middle, and, for the largest model on an 8 GB device, decode-collapsed past ~2,000 tokens—at which point prefill’s share falls not because prefill got cheap but because decode collapsed. Whether a deployment sees two regimes or three depends on the model a developer chooses; whether memory headroom is the reason is the next measurement. Any system that schedules, places or budgets on-device inference—and any benchmark that reports its speed—should measure the hardware and the context length together, rather than inherit an intuition from a datacentre it will never run in. It need not measure much: one calibration of ρ predicts the share, and the prediction’s failure announces the collapse.

REPRODUCIBILITY

Both harnesses, all raw per-run data, and the analysis scripts are public.

- CPU study
github.com/bapista/cpu-llm-latency
DOI 10.5281/zenodo.21765705
- Apple Silicon study
github.com/bapista/mlx-llm-latency
DOI 10.5281/zenodo.21786211

Every number in this paper traces to a line in a released results file. Earlier versions of both studies are available as preprints at the DOIs above.

REFERENCES

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 117–134.
- [2] Afsara Benazir and Felix Xiaozhu Lin. 2025. Profiling Large Language Model Inference on Apple Silicon: A Quantization Perspective. *arXiv preprint arXiv:2508.08531* (2025). arXiv:2508.08531 [cs.PF].
- [3] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *arXiv preprint arXiv:2305.05176* (2023).
- [4] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [5] Elias Frantar, Saleh Ashkboos, Torsten Hoeftler, and Dan Alistarh. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In *International Conference on Learning Representations (ICLR)*.
- [6] Georgi Gerganov and llama.cpp contributors. 2023. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>.
- [7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*.
- [8] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.
- [9] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*.
- [10] Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yanyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Tijmen Blankevoort, and Vikas Chandra. 2024. MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases. In *International Conference on Machine Learning (ICML)*.
- [11] Seonjin Na, Geonhwa Jeong, Byung Hoon Ahn, Jeffrey Young, Tushar Krishna, and Hyesoon Kim. 2024. Understanding Performance Implications of LLM Inference on CPUs. In *2024 IEEE International Symposium on Workload Characterization (IISWC)*. 169–180. <https://doi.org/10.1109/IISWC63097.2024.00024>
- [12] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. 2024. RouteLLM: Learning to Route LLMs with Preference Data. *arXiv preprint arXiv:2406.18665* (2024).
- [13] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*. arXiv:2311.18677.
- [14] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. 2022. Confident Adaptive Language Modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [15] Amir Taherin, Sana Taghipour Anvari, Charles Amante, Yixiao Chen, Ruben Noroian, Zlatan Feric, Nicolas Bohm Agostini, Pu Zhao, José Cano, Bin Ren, Yanzhi Wang, and David Kaeli. 2026. Hydra: Phase-Aware Workload Characterization of LLM Inference across Edge SoC Generations, Backends, and Quantization Levels. *arXiv preprint arXiv:2608.25053* (2026). To appear, IEEE IISWC 2026.
- [16] Jiajun Xu, Zhiyuan Li, Wei Chen, Qun Wang, Xin Gao, Qi Cai, and Ziyuan Ling. 2024. On-Device Language Models: A Comprehensive Review. *arXiv preprint arXiv:2409.00088* (2024).
- [17] Zhenliang Xue, Yixin Song, Zeyu Mi, Xinrui Zheng, Yubin Xia, and Haibo Chen. 2024. PowerInfer-2: Fast Large Language Model Inference on a Smartphone. *arXiv preprint arXiv:2406.06282* (2024).
- [18] Haolin Zhang and Jeff Huang. 2025. Challenging GPU Dominance: When CPUs Outperform for On-Device LLM Inference. *arXiv preprint arXiv:2505.06461* (2025). arXiv:2505.06461 [cs.DC].
- [19] Juntao Zhao, Jiuru Li, and Chuan Wu. 2025. Sandwich: Joint Configuration Search and Hot-Switching for Efficient CPU LLM Serving. *arXiv preprint arXiv:2507.18454* (2025). arXiv:2507.18454 [cs.AR].
- [20] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.
- [21] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. arXiv:2401.09670.